# DISCRETE MATHEMATICS

1. a) For each of the following statements, state whether it is true or false and provide a supporting proof.

   i) $\max\{100n^2, n^3\} = O(n^3)$. [ 3 ]

   *Answer:*
   *True.* $\max\{100n^2, n^3\} <= 100\max\{n^2, n^3\} = 100n^3 = O(n^3)$.

   ii) $\min\{100n^2, n^3\} = \Omega(n^3)$. [ 3 ]

   *Answer:*
   *False. For* $n > 100$, $\min\{100n^2, n^3\} = 100n^2$ *so* $\min\{100n^2, n^3\}/n^3 \to 0$ *as* $n \to \infty$ *and so* $\min\{100n^2, n^3\}$ *cannot be* $\Omega(n^3)$.

   iii) $n^3 \sin(n\pi/15) = \Theta(n^3)$. [ 3 ]

   *Answer:*
   *False. Whenever n is a multiple of 15,* $\sin(n\pi/15) = 0$ *which cannot be greater than some constant time* $n^3$.

   b) Give a tight bound for each of the following recurrence relations, or explain why it is not possible to do so.

   i) $T(n) = 8T(n/4) + O(n\sqrt{n})$. [ 2 ]

   *Answer:*
   $a = 8, b = 4, d = 1.5$ *so* $\log_b a = \log_4 8 = 1.5 = d$ *so* $T(n) = O(n^d \log n) = O(n^{1.5} \log n)$.

   ii) $T(n) = 9T(n/3) + n^2 + n$. [ 2 ]

   *Answer:*
   $a = 9, b = 3, d = 2$ *so* $\log_b a = \log_3 9 = 2 = d$ *so* $T(n) = O(n^d \log n) = O(n^2 \log n)$.

   iii) $T(n) = 4T(n/4) + 2n$. [ 2 ]

   *Answer:*
   $a = 4, b = 4, d = 1$ *so* $\log_b a = \log_4 4 = 1 = d$ *so* $T(n) = O(n^d \log n) = O(n \log n)$.

**Master Theorem.** If $T(n)$ satisfies

$$T(n) = a\,T(\,n/b\,) \;+\; O(n^d)$$

for some $a > 0$, $b > 1$ and $d \geq 0$, then

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

Answers

2. The quicksort algorithm sorts an array $X$ of $n$ elements in the following way. The last item in the array $X_{n-1}$ is called the *pivot*. The algorithm scans through the first $n-1$ items of the array dividing them into those less than or equal to the pivot value, and those larger than it. It then recursively sorts those two subarrays. Finally, it joins the sorted subarrays with the pivot value in the middle. In pseudocode:

```
function QUICKSORT(X)
    n = len(X)
    if n ≤ 1 then
        return X
    end if
    p = X_{n-1}
    L = array of elements X_i where X_i ≤ p and 0 ≤ i < n − 1
    R = array of elements X_i where X_i > p and 0 ≤ i < n − 1
    L = QUICKSORT(L)
    R = QUICKSORT(R)
    return join arrays L, [p], R
end function
```

Write $T(n)$ for the time it takes to run this code on an array of length $n$.

a) Let $k = \text{len}(L)$ be the length of the array of elements $X_i \leq p$. Write a recurrence equation for $T(n)$ with terms of the form $T(m)$ for $m < n$ and $\Theta(\cdots)$. [ 3 ]

*Answer:*

*The time taken to sort L will be $T(k)$. The size of R will be $n-1-k$ so sorting R will take $T(n-1-k)$. Partitioning the array into L and R takes $\Theta(n)$. So $T(n) = T(k) + T(n-1-k) + \Theta(n)$.*

b) If at each iteration, the algorithm divides $X$ into two equal halves $L$ and $R$, solve the recurrence relationship to find the time complexity of the algorithm. [ 3 ]

*Answer:*

*In this case the recurrence simplifies to $T(n) = 2T(n/2) + \Theta(n)$. We can solve this with the master method with $a = b = 2$ and $d = 1$ to get $T(n) = \Theta(n \log n)$.*

c) If at each iteration, the pivot $p = X_{n-1}$ is strictly greater than all other values $X_i$ ($0 \leq i < n-1$), solve the recurrence relationship to find the time complexity of the algorithm. [ 3 ]

*Answer:*

*In this case, the recurrence will be $T(n) = T(0) + T(n-1) + \Theta(n)$. Expanding, we get $T(n) = nT(0) + \Theta(n^2) = \Theta(n^2)$.*

d) Based on the above, what is the best and worst case performance of this algorithm? You should briefly explain your reasoning, but you do not need to provide a proof. [ 1 ]

*Answer:*

*Intuitively, divide and conquer is most efficient when the two partitions are equally sized, and least efficient when they are most disparately sized, so the best case will be $\Theta(n \log n)$ and the worst case $\Theta(n^2)$.*

Answers

3. We can represent an *operator* $x \oplus y$ in code by a function `f(x, y)`.

The *reduction* of an operator across an array of values $X$ of size $n$ is

$$X_0 \oplus X_1 \oplus X_2 \oplus \cdots \oplus X_{n-1},$$

or in code

```
f(X[0], f(X[1], f(..., f(X[n-2], X[n-1])...))).
```

You may assume that the operator is *associative* and *commutative*, so that the order of operations does not matter. Associative means $x \oplus (y \oplus z) = (x \oplus y) \oplus z$. Commutative means $x \oplus y = y \oplus x$.

a) Write code to compute the reduction of an operator defined by a function `f(x, y)` across an array $X$ with $n$ elements. Your code should use a loop, not recursion. [ 2 ]

*Answer:*
*The following Python code implements this.*

```
def reduction(f, X):
    n = len(X)
    if n<2:
        raise ValueError("Cannot reduce when n<2.")
    r = f(X[0], X[1])
    for i in range(2, n):
        r = f(r, X[i])
    return r
```

b) Compute the time complexity of your code, assuming each call `f(x, y)` is $O(1)$. [ 2 ]

*Answer:*
*Each call of $f$ is $O(1)$ and there are $n-1$ calls so the total cost is $O(n)$.*

c) Define and use a loop invariant to prove that your code is correct. [ 10 ]

*Answer:*
*The loop invariant in mathematical notation is $r = X_0 \oplus \cdots \oplus X_{i-1}$.*

***Initialisation.*** *At the first iteration of the for loop we have $i = 2$ and $r = X_0 \oplus X_1$ directly from the line above.*

***Maintenance.*** *If it's true at the start of the $i$th iteration, then $r = X_0 \oplus \cdots \oplus X_{i-1}$. During the loop we compute a new value $r = r \oplus X_i = X_0 \oplus \cdots \oplus X_i$. This is the value $r$ will hold at the start of the next loop, as required.*

***Termination.*** *At the end of the loop, we take $i = n$ and so we know that $r = X_0 \oplus \cdots \oplus X_{n-1}$, which is the value required.*

d) For each of the algorithms below, state whether or not they can be implemented as reductions of an operator. If they can be implemented as reductions, define the operator `f(x, y)`. If not, explain why (you do not need to write a proof).

i) The sum of all values in an array. [ 2 ]

*Answer:*
*It can be implemented with $f(x,y) = x + y$.*

ii) The smallest difference between two values in an array:

$$\min_{0 \le i < j < n} |X_i - X_j|$$

Answers

[ 2 ]

*Answer:*
*It cannot be implemented as a reduction because it needs to consider all pairs $(x,y)$ and not just adjacent pairs.*

iii) The maximum value in an array. [ 2 ]

*Answer:*
*It can be implemented with $f(x,y) = \max(x,y)$.*

e) Define and use the reduction of a non-commutative operator to write an algorithm to find the second largest value in an array $X$.

*Hint: the function `f(x, y)` should return a pair of values, `x` should be an element of the array and `y` a pair of values.* [ 5 ]

*Answer:*
*The following Python code implements this.*

```
def largest_two(x, y):
    largest, second = y
    if x<second:
        return y
    elif x<largest:
        return (largest, x)
    else:
        return (x, largest)

def second_largest(X):
    if len(X)<2:
        raise ValueError("Need at least 2 values")
    # initial value for second argument needs
    # to be a pair
    r = (-inf, -inf)
    for i in range(len(X)):
        r = largest_two(x[i], r)
    largest, second = r
    return second
```

Answers